

# 1-Factor Forcing Set in Product Graphs

J. Anitha<sup>1,\*</sup>, R. Manimegalai<sup>2</sup>, Akkuri Sruthi<sup>3</sup>

<sup>1</sup> Department of Mathematics

Rajalakshmi Institute of Technology

Chennai 600124, India

Email:anithaharish78@gmail.com

<sup>2,3</sup> Department of Computer Science and Engineering

PSG Institute of Technology and Applied Research

Coimbatore-641 062

<sup>2</sup>Email:drmm@psgitech.ac.in

## Abstract

A dynamically colour a graph  $G$  of vertices is to start with an initial subset  $S$  of coloured vertices and leave the other vertices uncoloured. A neighbour is coloured when a coloured vertex has exactly one non-coloured neighbour at each discrete time interval. The first set  $S$  is a zero forcing set (ZFS) of  $G$ . The zero forcing number (ZFN)  $\zeta(G)$  of a graph  $G$  is the smallest cardinality of a zero-forcing set of  $G$ .  $S$  is called a 1-factor forcing set of  $G$  if the initial set  $S$  has the additional condition that the subgraph induced by  $S$  is a perfect matching, or 1-factor. The minimal cardinality of a 1-factor forcing set of  $G$  is the 1-factor forcing number of  $G$ , or  $\zeta_{P_2}(G)$ . In this paper, we solve the 1-factor forcing number for product graphs of certain class and cactus-like graphs. Further, we propose algorithm to execute the zero-forcing number for certain networks.

**2020 Mathematics Subject Classification:** 05C69, 05C85, 05C90 and 05C20.

## 1 Introduction

A program of study on minimal ranks of matrices with certain combinatorial constraints includes the Zero Forcing problem in graphs (American Institute of Mathematics 2008).Analysing the bare minimum of actuators needed to run a network of dynamical systems was first proposed by the AIMs group.Inverse eigenvalue problems, PMU placement difficulties, and quantum control problems are all studied with a zero forcing set[2].In quantum dynamics and control theory of quantum mechanical systems, zero forcing is also known as *graph infection* or *graph propagation* [4]. This reality has been applied to computer science in the context of

---

\*Corresponding Author

fast-mixed searching, noise prevention, cooling, state preparation, and quantum state transfer by repeatedly applying a suitable quantum transformation [4].

The primary problem being addressed is the exploration of the behaviour and properties of zero forcing sets across different graph typologies. Specifically, this paper aims to investigate how zero forcing sets operate and influence the dynamics of graph evolution on certain graphs. The solution proposed involves a comprehensive study of zero forcing sets on various graph topologies. The paper contributes valuable insights and analyses that enhance our understanding of zero forcing sets and their behaviour across different graph structures. Various zero forcing sets are obtained for each graph through the proposed solution. The comparative study of zero forcing behaviour offers valuable insights into the impact of graph topology on zero forcing propagation, while theoretical contributions such as conjectures, theorems, and algorithms deepen our understanding of zero forcing theory. Additionally, the discussion of potential applications in practical domains highlights the relevance of zero forcing sets beyond theoretical graph theory, indicating potential avenues for real-world implementation and problem-solving.

**Colour-change rule:**[11] A vertex  $v$  should modify its colour to match  $u$  if it has precisely one neighbour  $u$  that has a different colour (zero or non-zero).

**Zero Forcing Set:** [11] A effective coloring of the nodes of a graph  $G$  starts with a leading subset  $S$  of colored nodes, with all remaining nodes being non-colored. A colored node that has exactly one non-colored neighboring node at each unique time interval makes this neighboring non-colored node colored. A ZFS of  $G$  is denoted by the leading set  $S$ . The minimal cardinality of a ZFS of  $G$  is the ZFN  $\zeta(G)$  of a graph  $G$ .

A network with sequence of nodes  $v_1, v_2, \dots, v_n$  and edges of  $\{v_i, v_{i+1}\}$ , where  $i = 1, 2, \dots, n-1$ , is called a path and  $n$  nodes  $v_1, v_2, \dots, v_n$  and edges  $\{v_1, v_2\}$ ,  $\{v_2, v_3\}$ ,  $\{v_3, v_4\}$ , and  $\{v_{n-1}, v_1\}$  make up a cycle  $C_n$ ,  $n \geq 3$ . A graph  $P_n^+$  that is created by connecting each path node to a single pendant edge is called a comb. A graph that has edges connecting each pair of distinct nodes is called a full graph  $K_n$ . The star graph  $K_{1,n}$  is a tree on  $n$  nodes with one node having node degree  $n-1$  and the other  $n-1$  having node degree 1.[12]

The Cartesian product  $G \square H$  is defined as:

- The node set of  $G \square H$  is the Cartesian product  $V(G) \times V(H)$ .
- Two nodes  $(x, y)$  and  $(x', y')$  are adjacent in  $G \square H$  if and only if either
  1.  $u = x'$  and  $y$  is adjacent to  $y'$  in  $H$ , or
  2.  $v = y'$  and  $x$  is adjacent to  $x'$  in  $G$ . [16]

A Strong product  $G \boxtimes H$  of graphs  $G$  and  $H$  is a graph such that:

1.  $u = v$  and  $u'$  is adjacent to  $v'$ , or
2.  $u' = v'$  and  $u$  is adjacent to  $v$ , or
3.  $u$  is adjacent to  $v$  and  $u'$  is adjacent to  $v'$ .

A ladder graph,  $L_n = P_n \times P_2$ , has one edge. The ladder graph  $L_n$  can be produced as the cartesian product of these two path graphs. Conversely,  $n$  copies of the cycle graph  $C_3$  can be joined with a common vertex, which becomes a universal vertex for the graph, to create the friendship graph  $F_n$ , also called the fan graph. There are  $3n$  edges and  $2n + 1$  nodes in  $F_n$ . [15] A graph without any cycles is called a tree. A binary tree is one in which every node can have a maximum of two descendants. A binary tree is considered complete if each internal node has precisely two descendants, which are the left and right offspring of the parent node. By attaching edges between the left and right children of the same parent node, the 1-rooted sister tree  $ST_r^1$  is created from the 1-rooted full binary tree  $T_r$ . [16]

## 2 Literature Review

The bounds of the ZFS of a graph  $G$ , in terms of its degrees, provide valuable insights into the inherent properties and dynamics of the graph. One crucial bound is the lower bound on the zero forcing number of a graph, denoted as  $Z(G)$ , which asserts that  $Z(G) \geq \delta - 1$ , where  $\delta$  represents the minimum degree of the vertices in  $G$ . This bound underscores a fundamental relationship between the structure of the graph and the size of its minimal forcing set. Intuitively, it suggests that the minimum forcing set size is influenced by the connectivity and degree distribution within the graph. When the minimum degree  $\delta$  is higher, it implies a greater level of connectivity and potential for vertex influence, thereby necessitating a larger ZFS to achieve full graph coloring. The propagation time, Chilakamarri et al. [3] refer to as the iteration index, has been calculated for a number of graph families, such as Cartesian products and different grid graphs.

In 2015, Davila et. al [9] has extended forcing problem to total forcing problem in graphs by way of restricting the structure of forcing sets as an induced subgraph. The lower and upper bounds on the total forcing number for each graph  $G$  are found to be  $2 \leq \zeta_t(G) \leq \zeta_c(G)$  in the same study.  $\zeta_t(T) \leq \frac{\Delta}{\Delta+2}n$  is an upper bound on the total forcing number of any tree  $T$  of order  $n \geq 3$  [10].

Zero Forcing Sets have applications beyond traditional graph theory, extending into the realm of quantum physics. In quantum systems, Zero Forcing Set principles are leveraged to model and control the propagation of quantum states and information through networks of qubits. By employing color-change rules akin to classical graph forcing, Zero Forcing Set algorithms can identify critical qubits whose manipulation induces desired transformations across the quantum network. This application of Zero Forcing Set in quantum physics holds promise for advancing quantum computing, communication, and cryptography, where efficient control over qubit interactions is essential for realizing the full potential of quantum technologies.

In network theory and control, Zero Forcing Sets play a vital role in optimizing network performance and resilience. By strategically selecting a minimal set of influencing nodes, Zero Forcing Set algorithms enable efficient monitoring and control of complex networked systems. Moreover, in communication networks, Zero Forcing Set techniques facilitate the identification of influential nodes for message propagation and routing, improving network efficiency and robustness against failures.

In biological and social networks, Zero Forcing Sets serve as activators for information spread and influence maximization. By identifying key nodes that can trigger cascading effects through the network, Zero Forcing Set algorithms provide insights into controlling epidemics, viral marketing campaigns, and social influence dynamics. In the domain of quantum network security, Zero Forcing Sets offer innovative approaches to detecting and mitigating quantum attacks. By analyzing the vulnerability of quantum communication networks to malicious actors, Zero Forcing Set algorithms can identify critical nodes whose compromise could disrupt network integrity or compromise sensitive information. This application of Zero Forcing Set in quantum network security holds significant implications for ensuring the privacy, authenticity, and resilience of quantum communication protocols, contributing to the development of secure quantum technologies for future communication infrastructures.

The diverse applications of Zero Forcing Sets across various disciplines underscore their significance and versatility in addressing complex problems in network theory, quantum physics, biological systems, and beyond. The literature reviewed in this study highlights the breadth and depth of research exploring Zero Forcing Set principles and algorithms, from their foundational concepts in graph theory to their practical applications in network control, quantum computing, and information spread dynamics.

Zero Forcing Set principles involve the identification of critical sets of vertices in a graph that can uniquely determine the state of the entire network. While initially studied for their implications in graph coloring and propagation phenomena, Zero Forcing Sets have since garnered significant attention for their practical applications in network control, quantum computing, epidemiology, and beyond. These studies have contributed to a deeper understanding of the fundamental properties of Zero Forcing Set, as well as their potential to address complex challenges in network analysis and optimization. Moreover, recent advancements in algorithmic techniques and computational methods have expanded the capabilities of Zero Forcing Set algorithms, enabling their application to increasingly large and complex network structures. This review of the literature offers a thorough summary of the state of the art in Zero Forcing Set research, emphasizing important discoveries, approaches, and new developments in the area.

A subset of vertices  $S$  of  $G$  is called a *zero forcing set* because it can colour all of  $G$  blue by repeatedly applying the colour change rule when the vertices in  $S$  are coloured blue and the remaining vertices are originally coloured white.  $\zeta(G)$  of  $G$  represents the zero forcing number, which is the lowest cardinality of a zero forcing set of  $G$ . In the event that  $S$  is a zero forcing set of  $G$  and possesses the further characteristic of having no isolated vertex in the subgraph of  $G$  that  $S$  induces, then  $S$  is a *total forcing set* of  $G$ .  $\zeta_t(G)$  represents the total forcing number of  $G$ , which is the minimal cardinality of a total forcing set of  $G$ . [9].

A set of edges in  $G$  such that no two edges in  $M$  are contiguous is called a matching  $M$  in  $G$ . If each vertex in  $G$  is incident to an edge in  $M$ , then a matching  $M$  of  $G$  is a perfect matching. Consequently, a 1-regular spanning subgraph of  $G$  is a perfect matching in  $G$ . It is also referred to as a 1-factor of  $G$  in the literature. If the induced subgraph  $G[S]$  has a perfect matching, then a zero forcing set  $S$  of  $G$  is a *1-factor forcing set* of  $G$ . The forcing number of  $G$  with the 1-factor, denoted  $\zeta_{P_2}(G)$ , is the minimum number of edges in  $G[S]$ , where  $S$  is minimum.

### 3 Product Graphs

In this section, we determine the lower bound for Cartesian product and Strong product of  $P_s \square P_r, P_s \square C_r$  and  $P_s \boxtimes P_r$ .

**Theorem 1.** *Let  $G$  be a Cartesian product of  $P_s \square P_r$ ,  $s \leq r, s, r \geq 4$ . Then*

$$\zeta_{P_2}(P_s \square P_r) = \begin{cases} \frac{s}{2} & \text{when } s \leq r \text{ and } s \text{ is even} \\ \lceil \frac{s}{2} \rceil & \text{when } s \leq r \text{ and } s \text{ is odd.} \end{cases}$$

*Proof.* Let  $B$  be a 1-factor forcing set of  $P_s \square P_r$ ,  $s$  and  $r$  are even. We claim that

$$\zeta_{P_2}(P_s \square P_r) = \begin{cases} \frac{s}{2} & \text{when } s \leq r \text{ and } s \text{ is even} \\ \lceil \frac{s}{2} \rceil & \text{when } s \leq r \text{ and } s \text{ is odd.} \end{cases}$$

**Case (i):** When  $s$  is even.

$\geq$ : Let  $B$  be a the set of blue edges in 1-factor forcing set of  $P_s \square P_r$ . We claim that  $|B| = \frac{s}{2}$ . Suppose not, we begin with the argument for Cartesian product of  $P_6 \square P_8$ . Let us assume,  $|B| < 3$ . The worst case arises when, by selecting two alternate edges in a 1-factor forcing set. Blue vertices in the iterative step 3 is adjacent to two white vertices. Further forcing is not possible, a contradiction. In general, By selecting  $\frac{s}{2} - 1$  alternate edges in column 1 then the vertex labelled  $(s - 2, 1)$  is adjacent to two white neighbours, a contradiction. See Figure 1(b) and 1(c). Therefore,  $|S| \geq \frac{s}{2}$ .

$\leq$ : Let  $B = \{(k, 1), (k + 1, 1)\} : 1 \leq k \leq s\}$  be the initial set of blue edges in 1-factor forcing set. We consider, the remaining vertices are in  $W = \{(k, l) : 1 \leq k \leq s, 2 \leq l \leq r\}$ ,  $W$  be the set of white vertices. By colour-change rule, initial set of vertices in  $B$  say,  $\{(k, 1), (k + 1, 1)\}$  is adjacent to exactly one white neighbour say,  $(k, 2)$ ,  $1 \leq k \leq s$  these vertices can be forced in 1<sup>st</sup> iterative step. The vertices in column 2 has exactly one white neighbours in column 3 say,  $(k, 3)$ ,  $1 \leq k \leq s$  can be forced in the second iterative step. By inductive process,  $(k, r - 1)$ ,  $1 \leq k \leq s$  forces  $(k, r)$ ,  $1 \leq k \leq s$ . All nodes in column  $r$  are colored blue. Thus,  $|S| \leq \frac{s}{2}$ . See Figure 1(d).

**Case(ii):** When  $s$  is odd.

$\geq$ : Let  $B$  be a the set of blue edges in 1-factor forcing set of  $P_s P_s$ . Let us consider, the Cartesian product  $P_5 P_8$ . Suppose  $|B| < 3$ . The worst case arises when, by selecting two alternate edges in a 1-factor forcing set. Blue vertices in the iterative step 3 is adjacent to two white vertices. Further forcing is not possible, a contradiction. In general, By selecting  $\lceil \frac{s}{2} - 1 \rceil$  alternate edges in column 1 then the vertex labelled  $(s - 1, 1)$  is adjacent to two white neighbours, a contradiction. Therefore,  $|S| \geq \lceil \frac{s}{2} \rceil$ .

$\leq$ : Let  $B = \{(k, 1), (k + 1, 1)\} \cup \{(s, 1), (s, 2)\} : 1 \leq i \leq s - 1\}$  be the initial set of blue edges in 1-factor forcing set. We consider, the remaining vertices are in  $W = \{(k, l) : 1 \leq k \leq s, 2 \leq l \leq r\}$ ,  $W$  be the set of white vertices. By colour-change rule, initial set of vertices in  $B$  say,  $\{(k, 1), (k + 1, 1)\} \cup \{(s, 1), (s, 2)\}$  is adjacent to exactly one white neighbour say,

$(k, 2)$ ,  $1 \leq k \leq s$  these vertices can be forced in  $1^{st}$  iterative step. The vertices in column 2 has exactly one white neighbours in column 3 say,  $(k, 3)$ ,  $1 \leq k \leq s$  can be forced in the second iterative step. By inductive process,  $(k, r - 1)$ ,  $1 \leq k \leq s$  forces  $(k, l)$ ,  $1 \leq k \leq s$ . All nodes in column  $l$  are colored blue. Thus,  $|S| \leq \lceil \frac{s}{2} \rceil$ .  $\square$

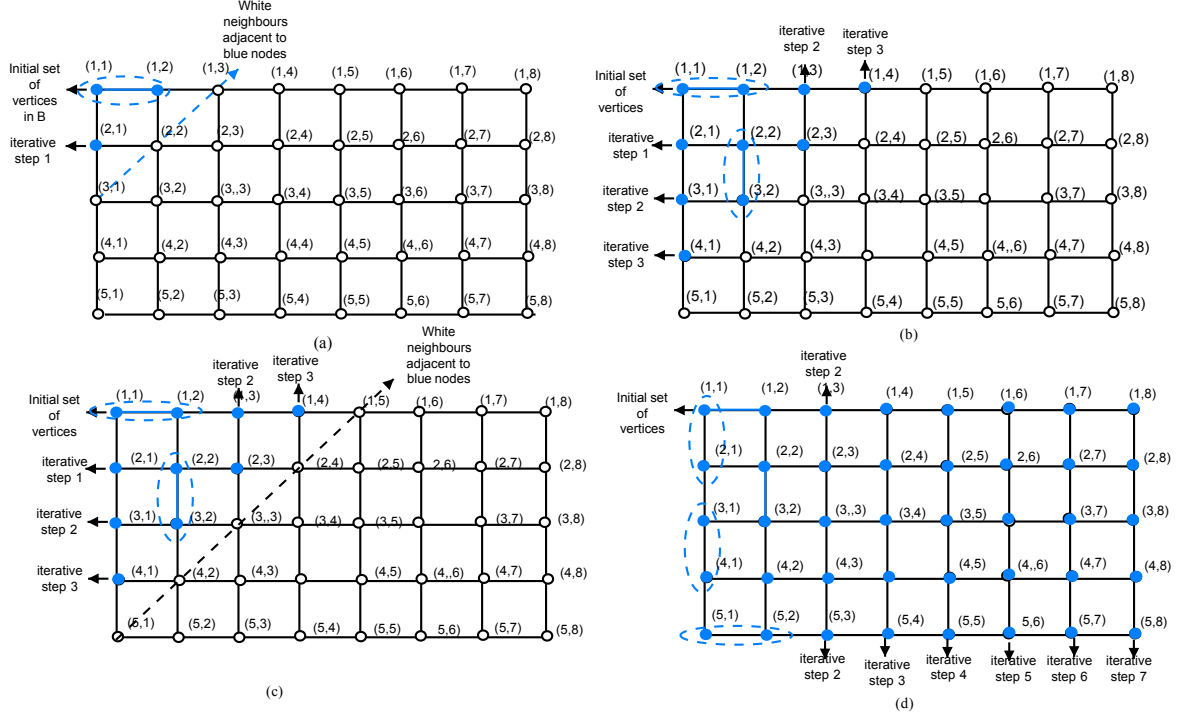


Figure 1: 1-Factor Forcing of Cartesian Product of  $P_5 \square P_8$  (i) Initial Set with  $|B| = 1$  (ii)

**Theorem 2.** Let  $G$  be a Cartesian product of  $P_s \square C_r$ ,  $s \leq r$ ,  $s, r \geq 4$ . Then

$$\zeta_{P_2}(P_s \square C_r) = \begin{cases} s & \text{when } s \leq r \text{ and } s \text{ is even} \\ \lceil s \rceil & \text{when } s \leq r \text{ and } s \text{ is odd.} \end{cases}$$

*Proof.* Let  $B$  be a zero forcing set of  $P_s \square C_r$ ,  $s$  and  $r$  are even. We claim that

$$\zeta_{P_2}(P_s \square P_r) = \begin{cases} s & \text{when } s \leq r \text{ and } s \text{ is even} \\ \lceil s \rceil & \text{when } s \leq r \text{ and } s \text{ is odd.} \end{cases}$$

. **Case (i):** When  $s$  is even.

$\geq$ : Let  $B$  be a the set of blue edges in 1-factor forcing set of  $P_s \square C_r$ . Let us consider, the Cartesian product  $P_6$

$square C_8$ . Suppose  $|B| < 6$ . The worst case arises when, by selecting two alternate edges in two consecutive columns in 1-factor forcing set. Blue vertices in the  $5^{th}$  row is adjacent to

two white vertices. Further forcing is not possible, a contradiction. In general, By selecting  $s - 1$  alternate edges in column 1 then the vertex labelled  $s - 1^{th}$  is adjacent to two white neighbours, a contradiction. See Figure 2(a). Therefore,  $S \geq s$ .

$\leq$ : Let  $B = \{\{(k, 1), (k + 1, 1)\} \cup \{(k, 2), (k + 1, 2)\} : 1 \leq k \leq s\}$  be the initial set of blue edges in 1-factor forcing set. We consider, the remaining vertices are in  $W = \{(k, l) : 1 \leq k \leq s, 3 \leq l \leq r\}$ ,  $W$  be the set of white vertices. By the rule of colour-change, initial set of vertices in  $B$  say,  $\{(k, 1), (k + 1, 1)\}$  and  $\{(k, 2), (k + 1, 2)\}$  is adjacent to exactly one white neighbour say,  $(k, r)$ ,  $1 \leq k \leq s$  and  $(k, r - 1)$ ,  $1 \leq k \leq s$  these vertices can be forced in  $1^{st}$  iterative step. The vertices in column 3 and column  $r$  has exactly one white neighbours in column 4 say,  $(k, 3)$ ,  $1 \leq k \leq s$  and column  $r - 1$  can be forced in the second iterative step. By inductive process,  $(k, \frac{r-1}{2})$ ,  $1 \leq k \leq s$  forces  $(k, \frac{r-1}{2})$ ,  $1 \leq k \leq s$  can be forced as blue.. All nodes in column  $r$  are colored blue. Thus,  $|S| \leq s$ .

**Case(ii) :** When  $s$  is odd.

$\geq$ : Let  $B$  be a the set of blue edges in 1-factor forcing set of  $P_s \square C_r$ . Let us consider, the Cartesian product  $P_5$

$squareC_8$ . Suppose  $|B| < 5$ . The worst case arises when, by selecting two alternate edges in two consecutive columns in 1-factor forcing set. Blue vertices in the  $5^{th}$  row is adjacent to two white vertices. Further forcing is not possible, a contradiction. In general, By selecting  $s - 1$  alternate edges in column 1 then the vertex labelled  $s - 1^{th}$  is adjacent to two white neighbours, a contradiction. See Figure 2(a). Therefore,  $S \geq s$ .

$\leq$ : Let  $B = \{\{(k, 1), (k + 1, 1)\} \cup \{(k, 2), (k + 1, 2)\} : 1 \leq k \leq s\}$  be the initial set of blue edges in 1-factor forcing set. We consider, the remaining vertices are in  $W = \{(k, l) : 1 \leq k \leq s, 3 \leq l \leq r\}$ ,  $W$  be the set of white vertices. By the rule of colour-change, initial set of vertices in  $B$  say,  $\{(k, 1), (k + 1, 1)\}$  and  $\{(k, 2), (k + 1, 2)\}$  is adjacent to exactly one white neighbour say,  $(k, r)$ ,  $1 \leq k \leq s$  and  $(k, r - 1)$ ,  $1 \leq k \leq s$  these vertices can be forced in  $1^{st}$  iterative step. The vertices in column 3 and column  $r$  has exactly one white neighbours in column 4 say,  $(k, 3)$ ,  $1 \leq k \leq s$  and column  $r - 1$  can be forced in the second iterative step. By inductive process,  $(k, \frac{r-1}{2})$ ,  $1 \leq k \leq s$  forces  $(k, \frac{r-1}{2})$ ,  $1 \leq k \leq s$  can be forced as blue.. All nodes in column  $r$  are colored blue. Thus,  $|S| \leq \lceil s \rceil$ .  $\square$

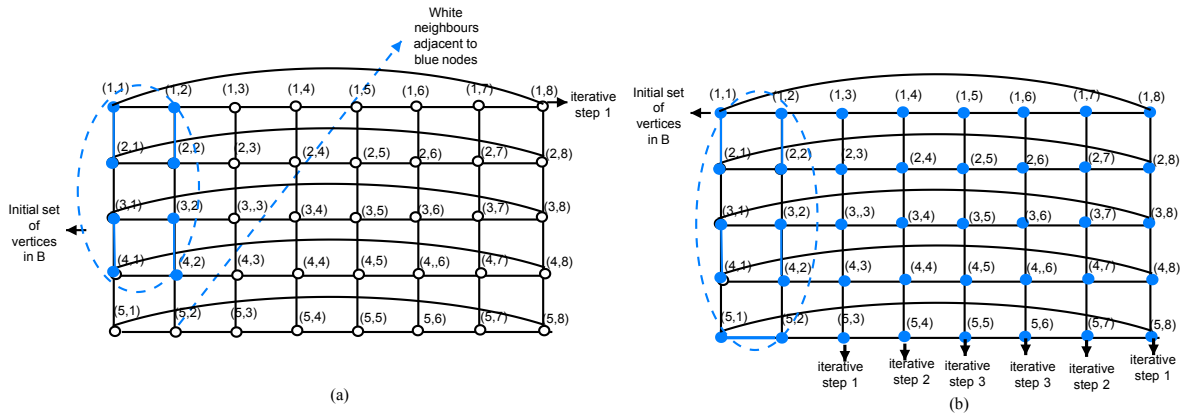


Figure 2: 1-Factor Forcing of Strong Product of  $P_6 \square P_8$

**Theorem 3.** Let  $G$  be a Strong product of  $P_s \square P_r$ ,  $s$  and  $r$  are even. Then  $\zeta_{P_2}(P_s \square P_r) = \frac{s+r}{2}$ .

*Proof.*  $\geq$ : Let  $B$  be a zero forcing set of  $P_s \boxtimes P_r$ ,  $s$  and  $r$  are even. We claim that  $|B| \geq \frac{s+r}{2}$ . Suppose  $|B| < \frac{s+r}{2}$ . Let  $B$  be a the set of blue edges in 1-factor forcing set of  $P_s \boxtimes P_r$ . Every row  $k$  and column  $l$  in  $G$  is adjacent to at least distinct two vertices in row  $k+1$  and column  $l+1$  of  $G$ . Let  $B = \{\{(k, 1), (k+1, 1)\} \cup \{(k, l), (k, l+1)\} : 1 \leq k \leq s, 2 \leq l \leq r-2\}$  be the initial set of blue edges in  $B$  then the remaining vertices are in  $W$ , be the set of white vertices. By the rule of colour-change, initial set of nodes in  $B$  cannot force the white nodes. Since every nodes in  $B$  has two white neighbouring vertices in column  $n-1$ . Thus,  $|B| \geq \frac{s+r}{2}$ . Therefore,  $\zeta_{P_2}(P_s \boxtimes P_r) = \frac{s+r}{2}$ .

$\leq$ : Let  $B$  be a the set of blue edges in 1-factor forcing set of  $P_s \boxtimes P_r$ . Let  $B = \{\{(k, 1), (k+1, 1)\} \cup \{(k, l), (k, l+1)\} : 1 \leq k \leq s, 2 \leq l \leq r-2\}$  be the initial set of blue edges in  $B$  then the remaining vertices are in  $W$ , be the set of white vertices. The set of blue edges in column 1 starts initial propagation. The vertex named  $(1, 1)$  forces  $(2, 2)$  in first step.  $(2, 1)$  forces  $(3, 2)$ , then  $(3, 1)$  forces  $(4, 2)$ . By inductive process,  $(s-1, 1)$  forces  $(s, 2)$ . All nodes in column 2 are colored blue. Then, blue nodes in column 2 is adjacent to two white neighbours. Further forcing not possible. Then add the node labelled  $(1, 3)$  in the initial set  $B$ . Now  $(1, 2)$  forces  $(2, 3)$  in the next step.  $(2, 2)$  forces  $(3, 3)$ , then  $(3, 2)$  forces  $(4, 3)$ . By inductive process,  $(s-1, 2)$  forces  $(s, 3)$ . All nodes in column 3 are colored blue. Then, blue nodes in column 3 is adjacent to two white neighbours. Again further forcing not possible. Then add the node labelled  $(1, 4)$  in the initial set  $B$ . Now  $(1, 3)$  forces  $(2, 4)$  in the next step.  $(2, 3)$  forces  $(3, 4)$ , then  $(3, 3)$  forces  $(4, 4)$ . By inductive process,  $(s-1, 3)$  forces  $(s, 4)$ . All nodes in column 4 are colored blue. By Inductively, blue nodes in column  $n-1$  is adjacent to two white neighbours in column  $n$ . Then add the node labelled  $(1, n)$  in the initial set  $B$ . Now  $(1, r-1)$  forces  $(2, r)$  in the next step.  $(2, r-1)$  forces  $(3, r)$ , then  $(3, r-1)$  forces  $(4, r)$ . By inductive process,  $(s-1, r-1)$  forces  $(s, r)$ . All nodes in column 4 are colored blue. Now  $B = \{\{(k, 1), (1, l)\} : 1 \leq k \leq s, 1 \leq l \leq r\}$  be the set of blue nodes forced remaining white nodes as blue. Hence  $|B| = \frac{s+r}{2}$ .  $\square$

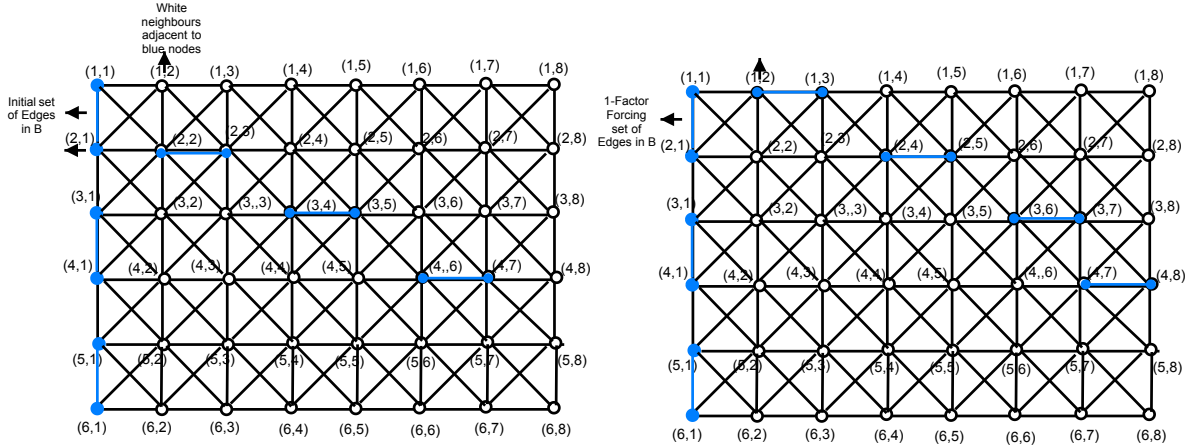


Figure 3: 1-Factor Forcing of Strong Product of  $P_6 \boxtimes P_8$

## 4 Experimental Results

In this section, we have proposed algorithm for zero forcing set of certain graphs such as linear graph, cyclic graph, star graph, ladder graph, comb graph, complete graph, binary tree and sibling tree. Here, all these algorithms work for graphs with  $n$  nodes and can compute their respective zero forcing sets. The zero forcing set algorithm for each graph type iterates through the nodes, gradually adding nodes to a forcing set until the set reaches a predefined size, typically corresponding to the graph's order. The outputs of these algorithms confirm their effectiveness and efficiency in identifying the minimum Zero Forcing Sets for each graph type. For each graph, we ran the algorithms with different initial conditions and configurations to validate their robustness. The results indicate that the algorithms consistently identified the correct Zero Forcing Set, with the expected number of forced vertices aligning with theoretical predictions. The experimental outputs underscore the adaptability of the algorithms to various graph structures, highlighting their potential for practical applications in network analysis and control.

### 4.1 ZFS Algorithm

```
function ZEROFORCING( $G$ )
  zeroForcingSet  $\leftarrow \{0, 1, \dots, n - 1\}$ 
  for  $i \leftarrow 1$  to  $n$  do {
    for each subset  $S(\{0, 1, \dots, n - 1\}, i)$  do {
      currentSet  $\leftarrow$  subset
      visited  $\leftarrow$  [False] *  $n$ 
      forces  $\leftarrow$  [0] *  $n$ 
      initialize  $\leftarrow$  True
      repeat
        if initialize then {
          for  $u := 1$  to  $n$  do {
            neighbors[ $u$ ]  $\leftarrow$  set of vertices adjacent to  $u$  in  $G$ 
          end for
          initialize  $\leftarrow$  False
        end if
        for each vertex  $u$  in currentSet with forces[ $u$ ] < degree( $u$ ) do {
          for each vertex  $v$  in neighbors[ $u$ ] do {
            if not visited[ $v$ ] then {
              visited[ $v$ ]  $\leftarrow$  True
              forces[ $v$ ]  $\leftarrow$  forces[ $v$ ] + 1
            end if
            if forces[ $v$ ] = degree( $v$ ) then {
              currentSet  $\leftarrow$  currentSet  $\cup$  { $v$ }
            }
          }
        }
      }
    }
  }
```

```

        end if
      end for
    end for
    until there does not exist a vertex  $v$  in currentSet with  $\text{forces}[v] < \text{degree}(v)$ 
  end for
  if if  $\text{SIZE}(\text{currentSet}) < \text{SIZE}(\text{zeroForcingSet})$  then{
    zeroForcingSet  $\leftarrow$  currentSet
  end if
end for
return zeroForcingSet

```

## 4.2 ZFS Algorithm of Cycle

```

function ZEROFORCINGSET(G)
  for for  $i$  from 0 to  $n - 1$  do {
    color[ $i$ ]  $\leftarrow$  0
  end for
  if if  $n \geq 2$  then {
    zfs  $\leftarrow$  [1, 2]
    repeat
      nextVertex  $\leftarrow$  FINDNEXT
      if if nextVertex  $\neq -1$  then {
        color[nextVertex]  $\leftarrow$  1
        zfs  $\leftarrow$  zfs  $\cup$  {nextVertex}
        for each neighbor  $u$  of nextVertex do{
          if color[ $u$ ] = 0 then{
            color[ $u$ ]  $\leftarrow$  -1
          end if
        end for
      end if
    until  $\exists v \in \text{zfs with color}[v] = 0$  end if
    until return zfs
  end function
  function FINDNEXT
    for for  $i$  from 0 to  $n - 1$  do {
      if if color[ $i$ ] = 0 and  $\text{len}(\text{graph}[i]) = 1$  then{
        return  $i$ 
      end if
    end for
    return -1
  end function

```

Output: [1,2]

### 4.3 ZFS Algorithm of Linear Graph

```
function ZEROFORCINGSET(G)
   $n \leftarrow \text{len}(\text{graph})$ 
   $\text{color} \leftarrow [0] * n$ 
   $\text{zfs} \leftarrow []$ 
   $\text{color}[0] \leftarrow 1$ 
   $\text{zfs} \leftarrow \text{zfs} \cup \{0\}$ 
  repeat
     $\text{nextVertex} \leftarrow \text{FINDNEXT}$ 
    if  $\text{nextVertex} \neq -1$  then {
       $\text{color}[\text{nextVertex}] \leftarrow 1$ 
       $\text{zfs} \leftarrow \text{zfs} \cup \{\text{nextVertex}\}$ 
      for for each neighbor  $u$  of  $\text{nextVertex}$  do {
        if  $\text{color}[u] = 0$  then {
           $\text{color}[u] \leftarrow -1$ 
        }
      }
    }
  until  $\exists v \in \text{zfs with } \text{color}[v] = 0$  return  $\text{zfs}[0:1]$ 
function FINDNEXT
  for  $i$  from 0 to  $n - 1$  do {
    if  $\text{color}[i] = 0$  and  $\text{len}(\text{graph}[i]) = 1$  then {
      return  $i$ 
    }
  }
  return  $-1$ 
```

**Output:**  $[0]$

### 4.4 ZFS Algorithm of Complete Graph

```
function ZEROFORCINGSET(G)
   $\text{forcingSet} \leftarrow \{\}$ 
  for  $i$  from 0 to  $\text{len}(\text{nodes}) - 1$  do {
     $\text{forcingSet} \leftarrow \text{forcingSet} \cup \{\text{nodes}[i]\}$ 
    if  $\text{size}(\text{forcingSet}) = 7$  then {
      break
    }
  }
```

```

    end for
    return forcingSet
Output: [0,1,2,3,4,5,6]

```

#### 4.5 ZFS Algorithm of Star Graph

```

function ZEROFORCING(G,initialSet)
    forcingSet  $\leftarrow$  initialSet
    remainingNodes  $\leftarrow$  set(range(len(graph))) - forcingSet
    repeat
        newForcingSet  $\leftarrow$  set()
        for for each node n in remainingNodes do {
            neighbors  $\leftarrow$  set(graph[n])  $\cap$  forcingSet
            if if size(neighbors) = 1 then {
                newForcingSet  $\leftarrow$  newForcingSet
            end if
        end for
        forcingSet  $\leftarrow$  forcingSet  $\cup$  newForcingSet
        remainingNodes  $\leftarrow$  remainingNodes - newForcingSet
    until remainingNodes is empty
    return forcingSet  $\cap$  set(range(8))
Output: [0,1,2,3,4,5,6,7]

```

#### 4.6 ZFS Algorithm of Fan Graph

```

function ZEROFORCINGSET(G(E))
    for for i from 1 to 10 with step 2 do {
        forcingSet  $\leftarrow$  forcingSet  $\cup$  {i}
    end for
    forcingSet  $\leftarrow$  forcingSet  $\cup$  {11}
    return forcingSet

```

**Output:** [1,3,5,7,9,11]

#### 4.7 ZFS Algorithm of Ladder Graph

```

function ZEROFORCINGSET(G(E))
    repeat
        for for each node v in graph.nodes() with v not in forcingSet do {

```

```

    neighbors ← set(graph.neighbors(v))
    if neighbors is superset of forcingSet then{
        forcingSet ← forcingSet ∪ v
        changed ← True
    end if
end for
if not changed then {
    break
end if
until ∃ v not in forcingSet such that the neighbors of v are a superset of forcingSet
return forcingSet

```

**Output:** Zero Forcing Sets (Ladder Graph, 12 Nodes): {11}, {10}, {10, 11}

#### 4.8 ZFS Algorithm of Comb Graph

```

function ZEROFORCINGSET(G(E), desiredZFSet)
    for for each node v in desiredZFSet do {
        if if G has a node v then {
            zfSet ← zfSet ∪ {v}
        end if
    end for
    return zfSet

```

**Output:** Zero Forcing Set (Comb Graph, 12 Nodes): {1, 2, 3, 4, 5, 6}

#### 4.9 ZFS Algorithm of Path Graph

```

function ZEROFORCINGSET(G(E))
    for for i from 0 to n - 1 do {
        color[i] ← 0
    end for
    zfs ← []
    color[0] ← 1
    zfs ← zfs + [0]
    for for i from 1 to n - 2 do {
        if if color[i] == 0 and color[i - 1] == 1 and color[i + 1] == 1 then {
            color[i] ← 1
            zfs ← zfs + [i]
        end if
    end for

```

```
return zfs
```

**Output:** Minimum ZFS: [1,6,7]

#### 4.10 ZFS Algorithm of Binary Tree

```
function ZEROFORCINGSET(G(E))
    forcingSet ← leafNodes
    repeat
        for for each node  $v$  in graph.nodes() with  $v$  not in forcingSet do {
            neighbors ← set(graph.neighbors( $v$ ))
            if if neighbors is superset of forcingSet then{
                forcingSet ← forcingSet  $\cup$  { $v$ }
                Added  $v$  to forcing set
            end if
        end for
    until  $\exists v$  not in forcingSet such that the neighbors of  $v$  are a superset of forcingSet
    return forcingSet
```

**Output:** ZFS (Complete Binary Tree, Level 4 with Specific Leaf Nodes): {16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31}

#### 4.11 ZFS Algorithm of Sibling Tree

```
function ZEROFORCINGSET(G(E))
    forcingSet ← leafNodes
    repeat
        for for each node  $v$  in graph.nodes() with  $v$  not in forcingSet do {
            neighbors ← set(graph.neighbors( $v$ ))
            if if neighbors is superset of forcingSet then{
                forcingSet ← forcingSet  $\cup$  { $v$ }
                Added  $v$  to forcing set
            end if
        end for
    until  $\exists v$  not in forcingSet such that the neighbors of  $v$  are a superset of forcingSet
    return forcingSet
end function
function TOBINARYTREE(tree)
    for for each node  $n$  in tree.nodes() with len(tree.neighbors( $n$ )) > 2 do {
        binaryTree[ $n$ ] ← [tree.neighbors( $n$ )[0], tree.neighbors( $n$ )[1]]
        for for each child  $c$  in tree.neighbors( $n$ )[2:] do{
```

```

        binaryTree[len(binaryTree)]  $\leftarrow$   $[c]$ 
    end for

    binaryTree[ $n$ ]  $\leftarrow$  tree.neighbors( $n$ )
end for
return binaryTree
Output: ZFS (Binary Tree): {16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30,
31}

```

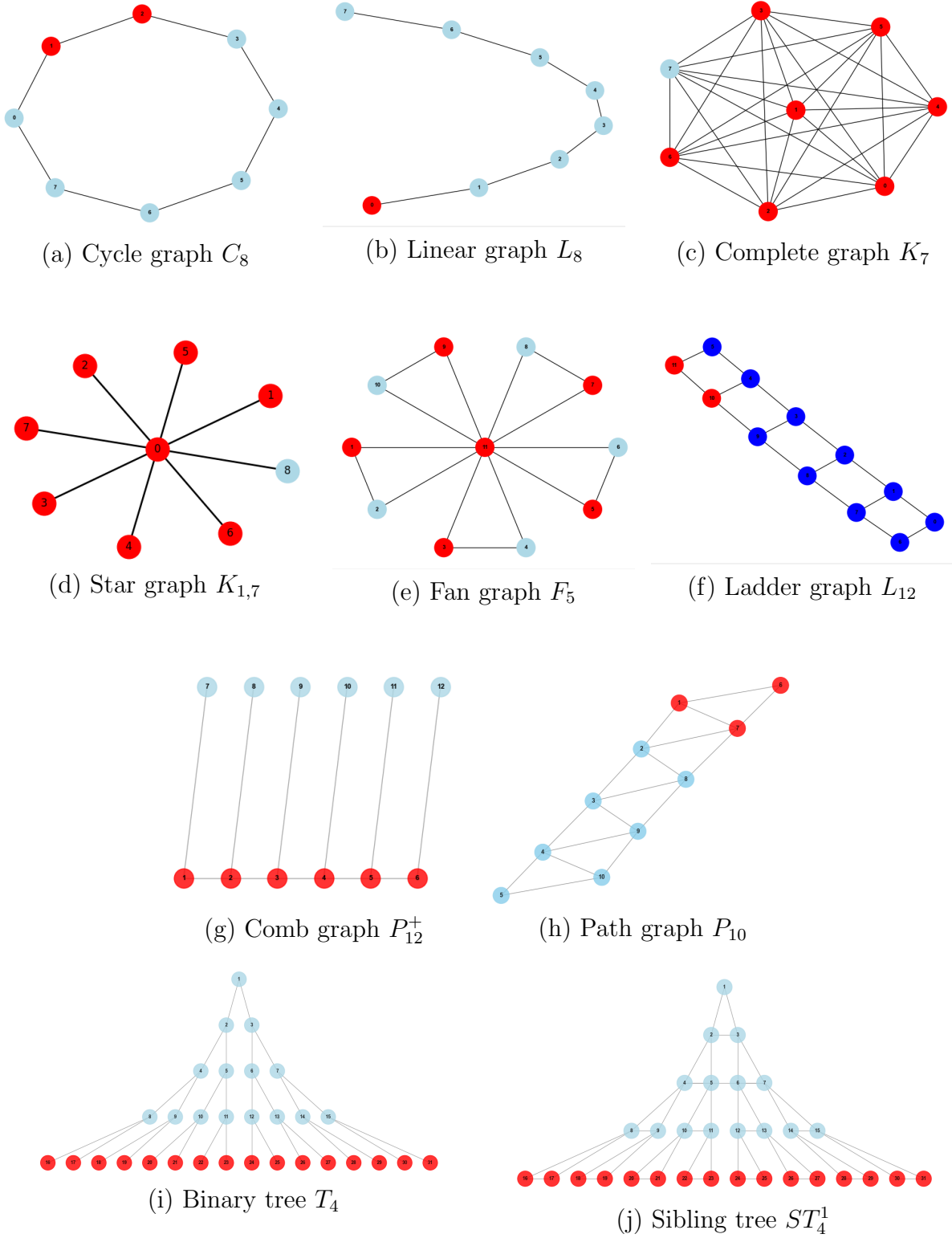


Figure 4: The red color vertices indicates the zero forcing sets of various graphs

Zero Forcing Algorithms enable efficient computation of zero forcing properties, facilitating the practical implementation of zero forcing techniques in applications such as network control, graph colouring and community detection. Empirical validation enhances the credibility and robustness of zero forcing techniques, making them more reliable for practical applications.

## 5 Conclusions

In this paper, we have developed and implemented algorithms to determine the minimum size of a Zero Forcing Set for a variety of graph structures, including cyclic graph, linear graph, complete graph, star graph, fan graph, ladder graph, comb graph, path graph, binary tree, and sibling tree. By leveraging the unique characteristics of each graph type, our algorithms demonstrate both efficiency and adaptability, providing practical tools for network analysis across diverse configurations. The concept of a Zero Forcing Set is integral to understanding how influence, control, or information can propagate through a network. By applying color-changing rules, a Zero Forcing Set ensures that the entire graph can be forced starting from an initial set of vertices.

For each graph type, the algorithms have been tailored to exploit their specific structural properties. In star graphs  $K_{1,n}$ , the central vertex plays a critical role in the Zero Forcing Set. The unique hub-and-spoke structure allows the central vertex to force all peripheral vertices, making it a highly efficient control point. For complete graphs  $K_n$  we have demonstrated that the Zero Forcing Set is straightforward, consisting of any  $n - 1$  vertices due to the graph's high connectivity. This results in efficient computation and straightforward application. In cyclic graphs, the cyclical nature requires a more nuanced approach to identify the minimal forcing set. The linear and path graphs, with their straightforward connectivity, allow for direct application of forcing rules, whereas fan and comb graphs introduce more complexity with their combined structures. The algorithms for binary and sibling trees take advantage of hierarchical relationships to efficiently propagate the forcing process through the network.

The comprehensive set of algorithms that have been presented in this study significantly advances the understanding and practical application of zero forcing sets in various graph structures. By offering efficient methods for identifying Zero Forcing Set in a wide array of graphs, it provides a robust foundation for future research and development. Continued exploration and enhancement of these algorithms will further support the creation of resilient and efficient network monitoring and control systems, contributing to more secure and sustainable network infrastructures.

## References

- [1] A. Aazami, Domination in graphs with bounded propagation: Algorithms, formulations and hardness results, *Journal of Combinatorial Optimization* 19 (2010) 429–456.

- [2] AIM Special Work Group, Zero forcing sets and the minimum rank of graphs. *Linear Algebra Appl.* 428(7), 1628-1648 (2008).
- [3] F. Barioli, W. Barrett, S. Fallat, H. Hall, L. Hogben, H. van der Holst, B. Shader, Zero forcing parameters and minimum rank problems, *Linear Algebra Appl.* 433 (2010) 401–411.
- [4] D. Burgarth, V. Giovannetti, L. Hogben, S. Severini, and M. Young, ‘Logic circuits from zero forcing’, *Natural Computing* 14(3), (2015) 485-490.
- [5] D. Burgarth, V. Giovannetti Full control by locally induced relaxation *Physical Review Letters*, 99 (2007), pp. 100-501
- [6] K.B. Chilakamarri, N. Dean, C.X. Kang, E. Yi Iteration Index of a Zero Forcing Set in a Graph *Bulletin of the Institute of Combinatorics and its Applications*, 64 (2012), pp. 57-72
- [7] S. Severini Nondiscriminatory propagation on trees *Journal of Physics A*, 41 (2008), p. 482002 (Fast Track Communication)
- [8] M. Gentner, L.D. Penso, D. Rautenbach and U.S. Souza, Extremal values and bounds for the zero forcing number, *Discrete Applied Mathematics*, 214, (2016) 196-200.
- [9] R. Davila, and M.A. Henning, On total forcing number of a graph, *Discrete Applied Mathematics*, 257, (2019) 115-127.
- [10] R. Davila, and M.A. Henning, Total forcing in trees, *Discussiones Mathematicae Graph Theory*, 40(3), (2017) 1-28.
- [11] J. Beier, J. Fierson, R. Haas, H. Russell, K. Shavo, Classifying coloring graphs, *Discrete Mathematics* 339 (2016) 2100–2112.
- [12] J.M. Xu, Topological structure and analysis of interconnection networks, Kluwer Academic Publisher, Netherlands, (2001).
- [13] Sandi Klavžar, I. Peterin, S. S. Zemljic, Hamming dimension of a graph—The case of Sierpiński graphs, *European Journal of Combinatorics*, 34 (2013) 460-473.
- [14] R. Sundara Rajan, J. Anitha, Indra Rajasingh, 2-Power Domination in Certain Interconnection Networks, *Procedia Computer Science* 57 ( 2015 ) 738 – 744.
- [15] J. Anitha and I. Rajasingh, 1-Factor Forcing in Certain Interconnection Networks, *Procedia Computer Science*, 172 (2020) 187-192.
- [16] K. Benson, D. Ferrero, M. Flag, V. Furst, L. Hogben, V. Vasilevska, B. Wissman, Zero forcing and power domination for graph products, *Australas. Journal of Combinatorics* 70 (2018) 221–235.
- [17] B. Brimkov, C.C. Fast, I.V. Hicks, Computational approaches for zero forcing and related problems, *European Journal of Operational Research* 273 (2019) 889–903.